



Project presentation

By Sofia Ong & Shantelle



addict

To create a simple daily planner web application that allows users to add tasks and events, track their mood, and receive reminders.

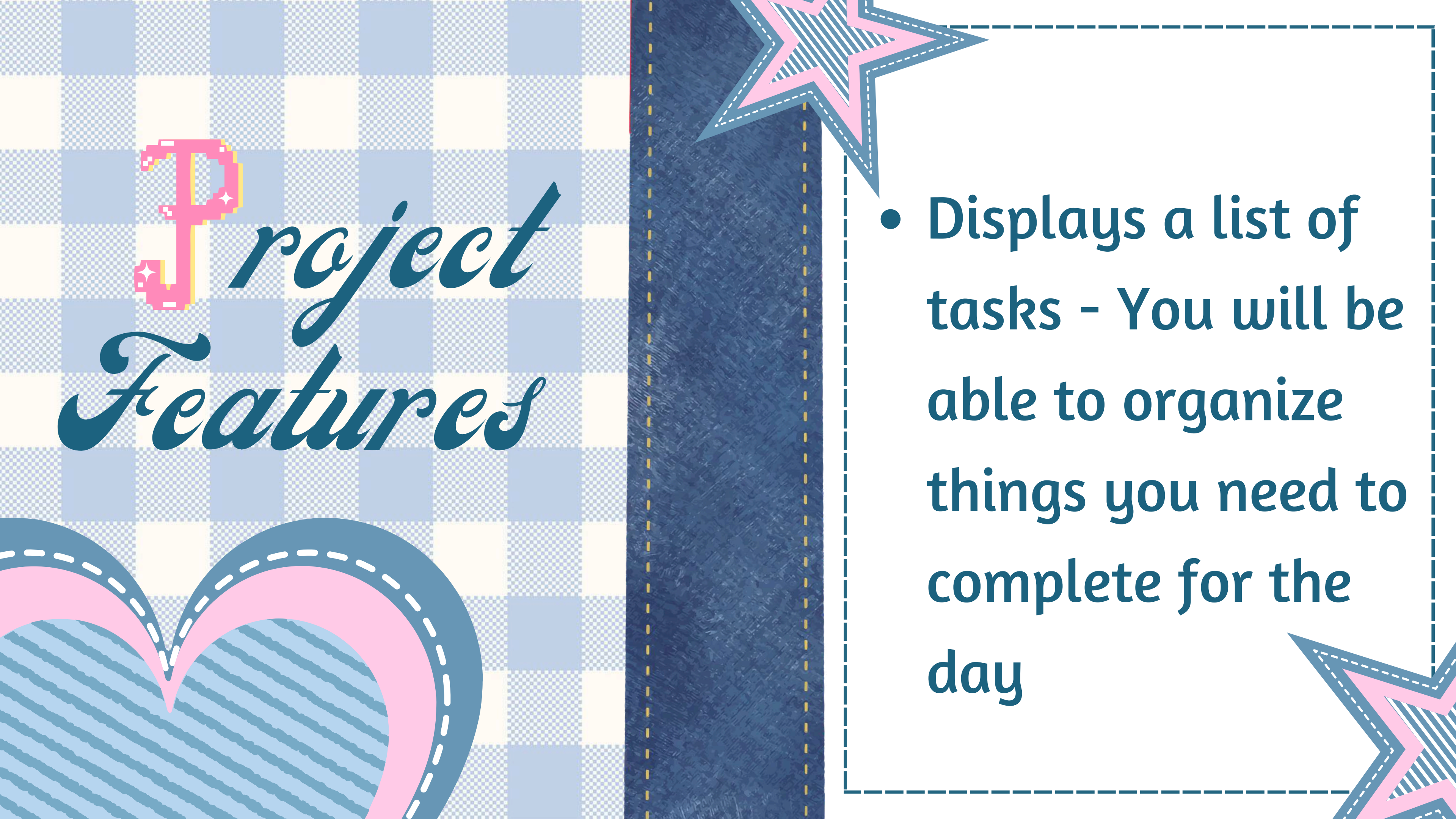
Project Objectives





lanned Features

- An alarm that notifies you whether your allotted time for a task is finished



Project Features

- Displays a list of tasks - You will be able to organize things you need to complete for the day

Methodology

- * Multi-step interface

Implemented using separate HTML sections (.step) and controlled with JavaScript (goToStep(n)), which toggles visibility via CSS classes.

- * Event management

Events stored in an array (events[]) with name, date, time

Added via form inputs → validated → pushed to array → rendered using DOM manipulation.

- * Task management

Tasks stored in tasks[] with name and time (minutes).

Similar process: input → validation → storage → dynamic rendering.

- * Data persistence

Entire data stored in one object and saved using localStorage (JSON.stringify).

Loaded on startup using JSON.parse.

- * Reminder system

Implemented using setTimeout():

- * Events → delay based on actual date/time

- * Tasks → delay based on minutes

Triggers notification, sound (Audio API), and alert.

- * Dynamic UI updates

Lists updated using createElement() and innerHTML after every change.

Methodology

- * HTML

Provides structure for multi-step form and content sections.

- * CSS

Used for layout, styling, and visibility control (e.g., .active class).

- * JavaScript

Handles all logic: navigation, validation, storage, scheduling, and rendering.

- * localStorage

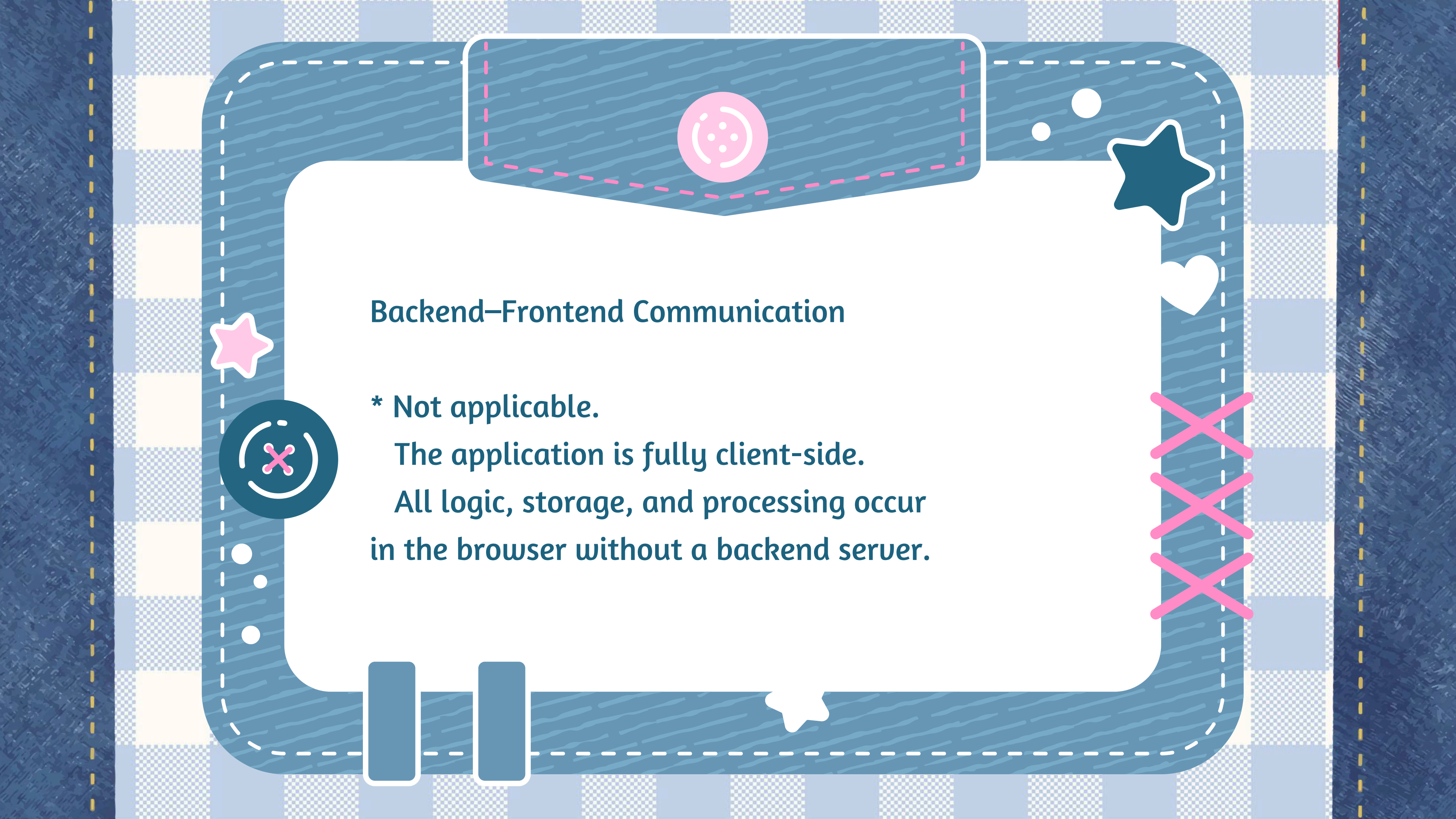
Chosen for persistent client-side storage without requiring a server.

- * Notification API

Enables real-time reminders directly in the browser.

- * Web Audio API

Provides sound alerts without external files.



Backend–Frontend Communication

* Not applicable.

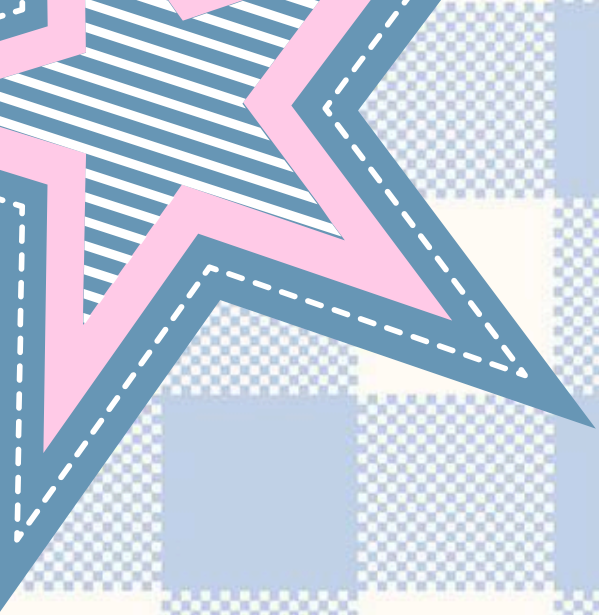
The application is fully client-side.
All logic, storage, and processing occur
in the browser without a backend server.

Methodology

- * Client-side only architecture
 - * Advantage: Simple, no server required, works offline
 - * Trade-off: Data limited to one device/browser, no synchronization
- * localStorage instead of database
 - * Advantage: Easy to implement
 - * Trade-off: Limited storage and no advanced querying
- * setTimeout for scheduling
 - * Advantage: Simple timer implementation
 - * Trade-off: Timers reset if page is closed or refreshed
- * Step-based UI
 - * Advantage: Reduces user overload, structured flow
 - * Trade-off: Less flexibility for quick navigation between sections

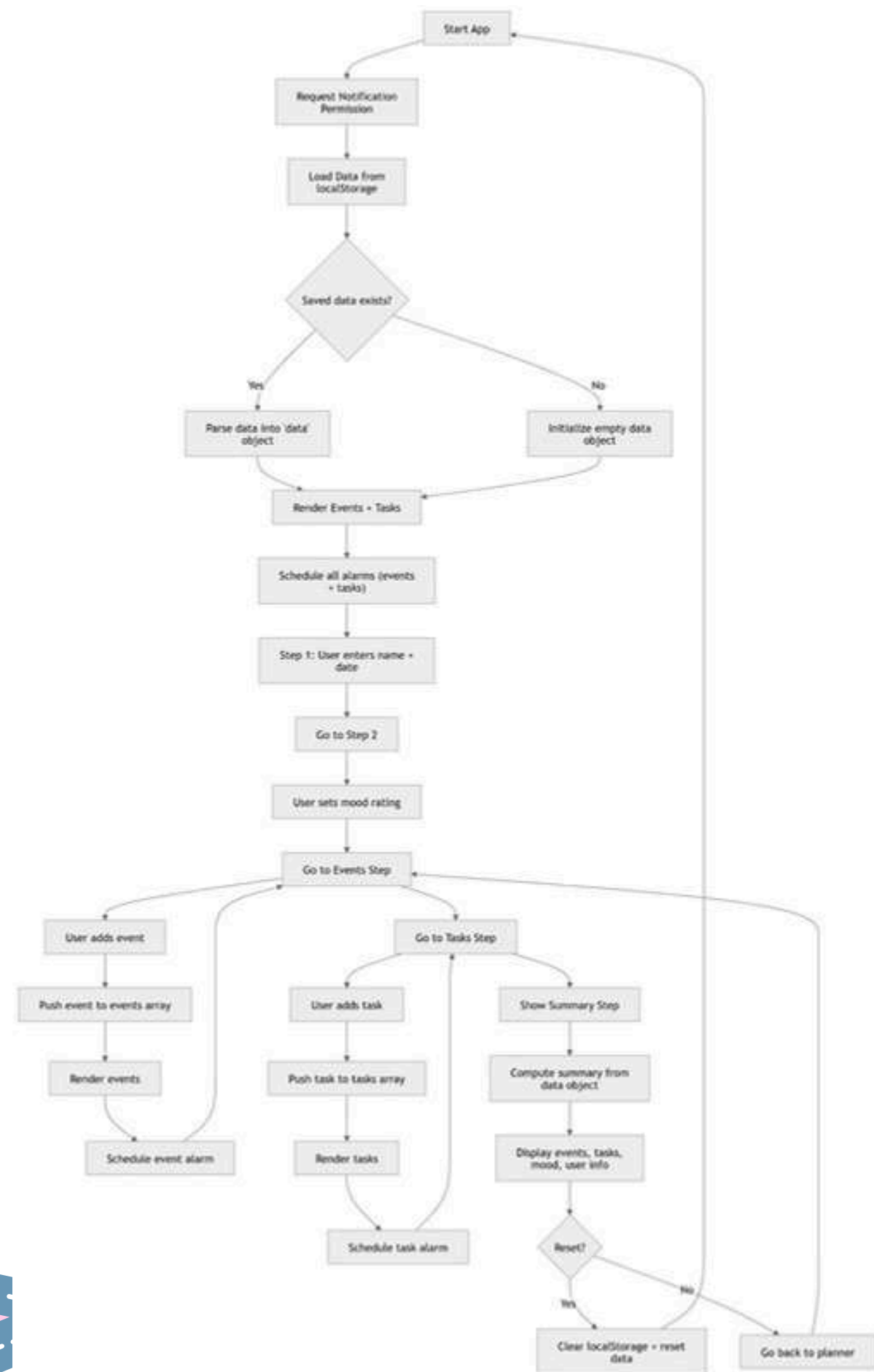
Methodology

- * User privacy
 - * All data stored locally in browser
 - * No data transmission or external storage
- * Data security
 - * No sensitive data collected (only name, tasks, events)
- * Accessibility
 - * Simple layout and readable inputs
 - * Limitation: No advanced accessibility features (e.g., screen reader optimization not explicitly implemented)
- * User control
 - * Users can delete individual items or reset all data
 - * Confirmation required before full reset to prevent accidental loss



Process

Flow





Thank
You!